untitled

File Edit Format Run Options Window Help

```
from itertools import permutations

def tour_length(dist, tour):
    return sum(dist[tour[i]][tour[(i + 1) % len(tour)]] for i in range(len(tour)))

# 1. Brute force (exact)
def tsp_brute_force(dist):
    n = len(dist)
    best_tour, best_cost = None, float("inf")
    for perm in permutations(range(1, n)):
        tour = (0,) + perm
        cost = tour_length(dist, tour)
        if cost < best_cost:
            best_tour, best_cost = tour, cost
    return list(best_tour) + [0], best_cost

# 2. Held-Karp dynamic programming (exact)
def tsp_held_karp(dist):
    n = len(dist)
    INF = float("inf")
    dp = {(1 << j, j): (dist[0][j], 0) for j in range(1, n)}
    for mask in range(1, 1 << n):
        if mask & 1:
            continue
        for j in range(1, n):
            if not mask & (1 << j) or mask == (1 << j):
                continue
            prev_mask = mask ^ (1 << j)
            best = (INF, None)
            for k in range(1, n):
                if prev_mask & (1 << k):
                    cost = dp[(prev_mask, k)][0] + dist[k][j]
                    if cost < best[0]:
                        best = (cost, k)
            dp[(mask, j)] = best
    full = sum(1 << j for j in range(1, n))
    best_cost, last = INF, None
    for j in range(1, n):
        cost = dp[(full, j)][0] + dist[j][0]
        if cost < best_cost:
            best_cost, last = cost, j
    order, mask, j = [], full, last
    while j != 0:
        order.append(j)
        prev = dp[(mask, j)][1]
        mask ^= (1 << j)
        j = prev
```

Ln: 77 Col: 54

26°C Light rain

ENG IN 09:34 25-10-2026

```
Brute force       : A -> B -> D -> C -> A   cost = 80
Held-Karp (DP)    : A -> C -> D -> B -> A   cost = 80
Nearest neighbour: A -> B -> D -> C -> A   cost = 80
```